Università
degli Studi
della Campania
Luigi Vanvitelli

15 Maggio 2025

Classificazione di oggetti astronomici: un approccio basato sul Machine Learning

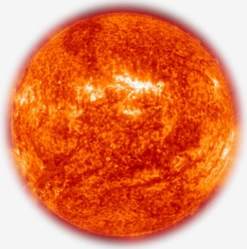
Laurea in Ingegneria Elettronica e Informatica

Gaetano Improta – A13/2647

Relatore: prof. Antonio Esposito

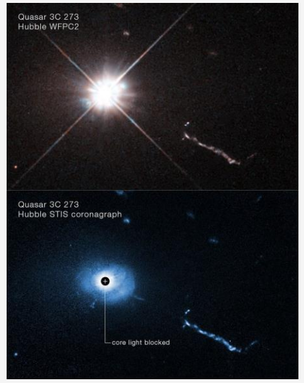
Contesto scientifico

Universo osservabile



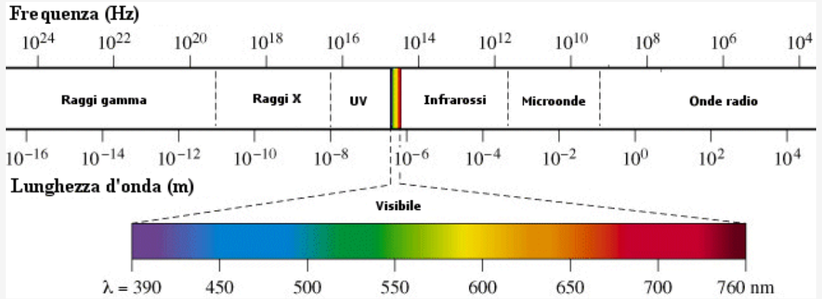
Stelle

Galassie

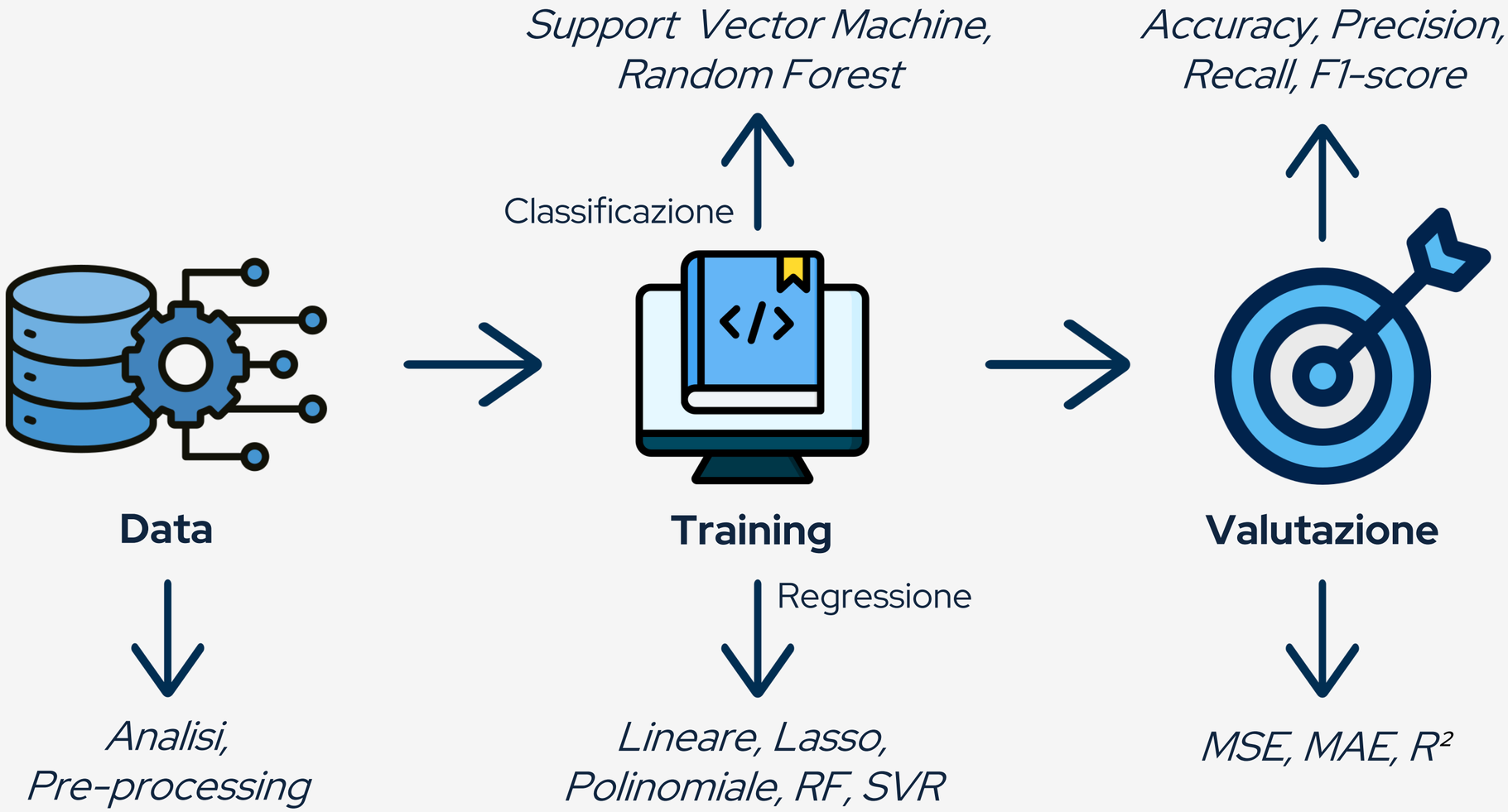


Quasar

Fotometria



Machine Learning supervisionato



Obiettivi

01 Photometric Classification (Star, Galaxy, Qsos)

- Analisi dei dati fotometrici
- Implementazione dei modelli di classificazione
- Ottimizzazione
- Confronto

02 Redshift Regression

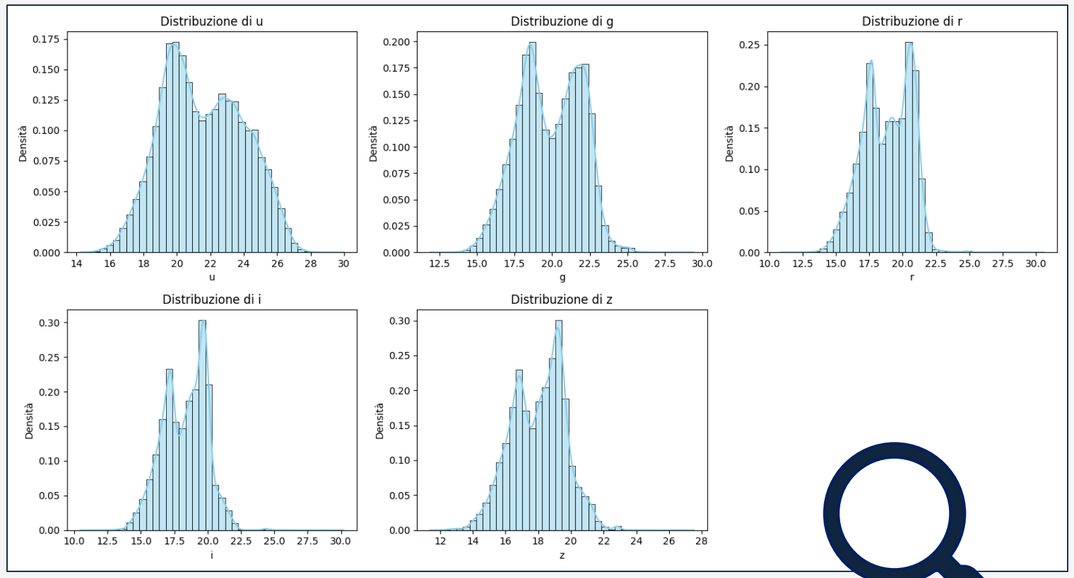
- Suddivisione dei dati classificati
- Applicazione dei modelli per classes
- Analisi e confronto
- Limiti

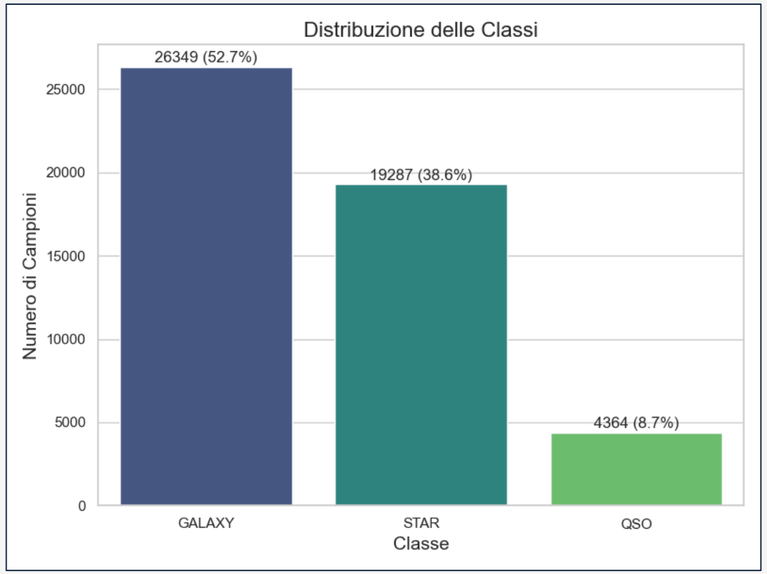
Il dataset

Progetto che ha raccolto dati di numerosissimi oggetti astronomici in cinque bande fotometriche.

Bande fotometriche
u : ultravioletto
g : verde-azzurra
r : rossa
i : vicino infrarosso
z : infrarosso

Distribuzione delle feature





Notiamo uno **sbilanciamento tra le classi**

Fase di apprendimento più delicata!

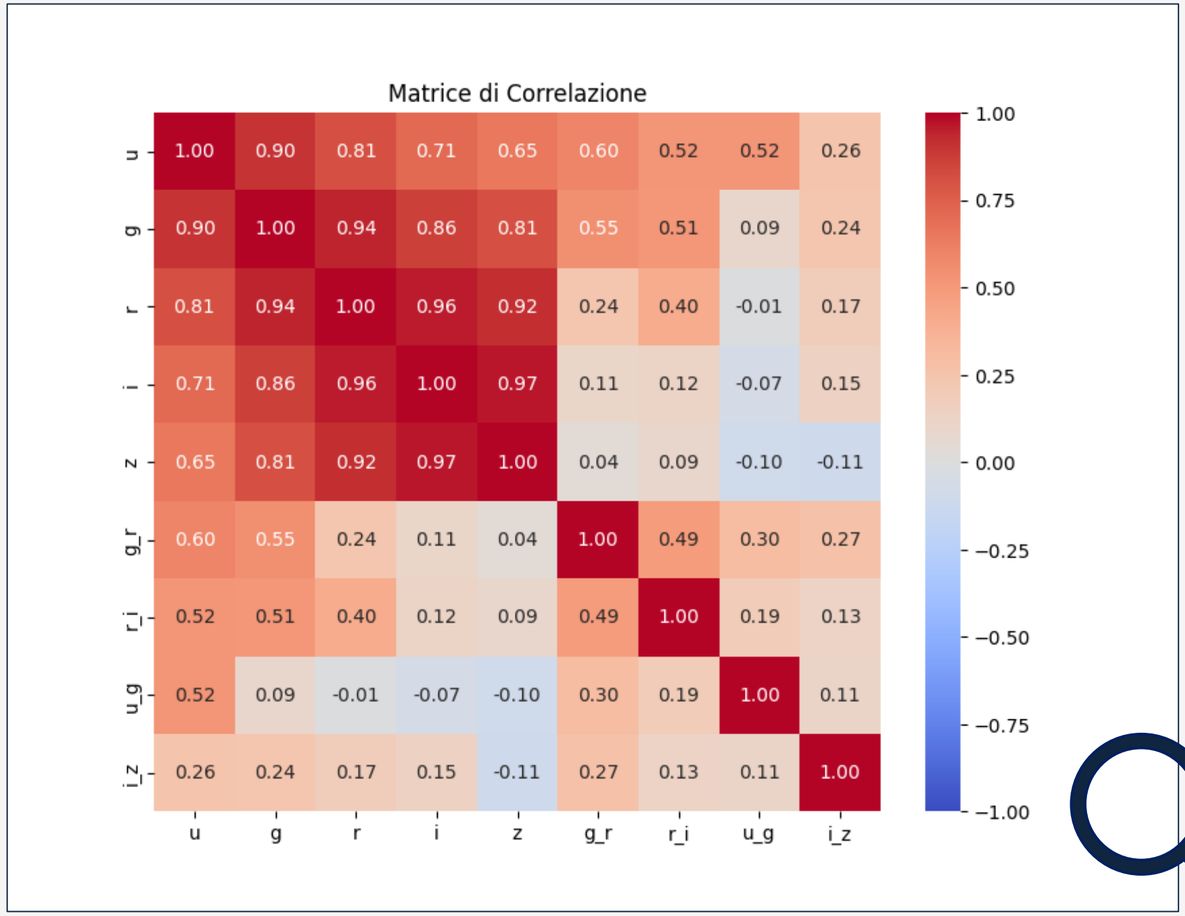
Feature Engineering

Derivazione degli
indici di colore

aggiunta alle sole magnitudini per migliorare la classificazione

Indici di colore
u - g
g - r
r - i
i - z

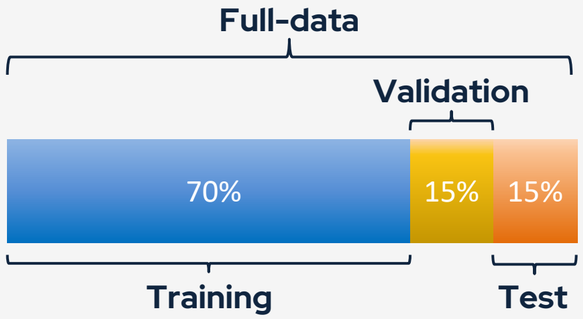
Analisi delle **correlazioni** tra le feature



Pre-processing

Import del dataset

```
import pandas as pd
df = pd.read_csv("sdss_data_big.csv", header=1)
```



Suddivisione in train-validation-test

```
from sklearn.model_selection import train_test_split
X_train_val, X_test, y_train_val, y_test = train_test_split(X, y_class, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val, test_size=0.3, random_state=42)
```

Standardizzazione

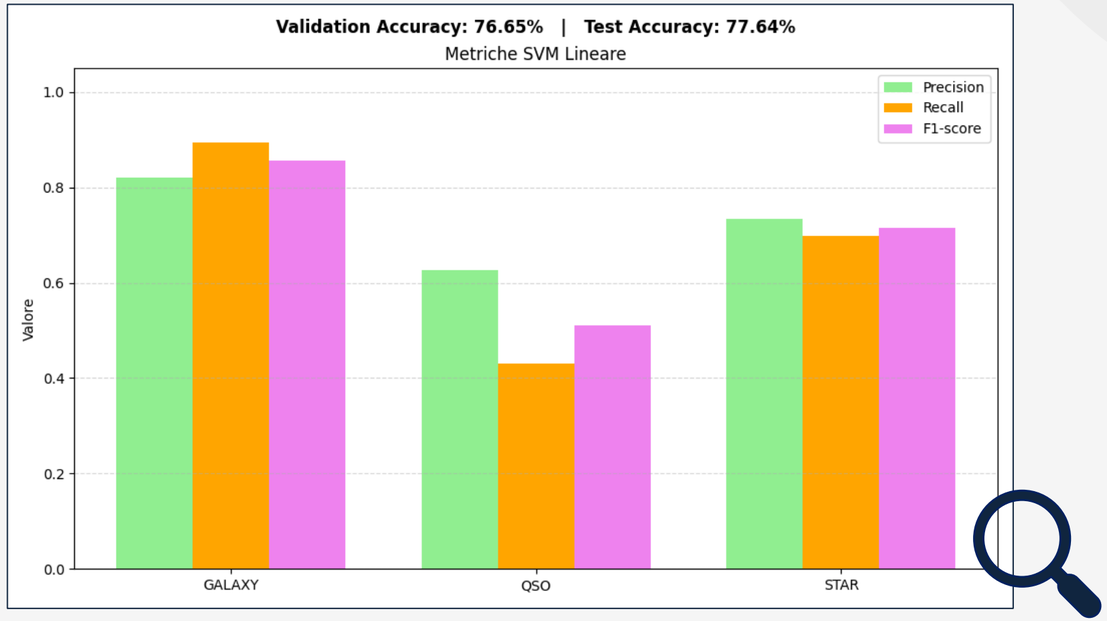
```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

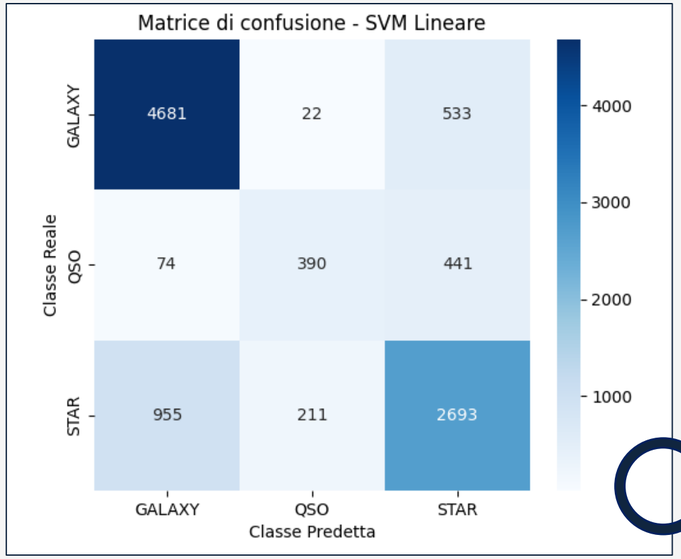
$$x' = \frac{x - \mu}{\sigma}$$

SVM lineare

Utilizzata come **baseline**

```
from sklearn.svm import SVC
model = SVC(kernel='linear', decision_function_shape='ovo', random_state=42)
model.fit(X_train_scaled, y_train) #Training del modello
y_val_pred = model.predict(X_val_scaled)
y_test_pred = model.predict(X_test_scaled)
```





Validation-accuracy: 76.7%
Test-accuracy: 77.6%

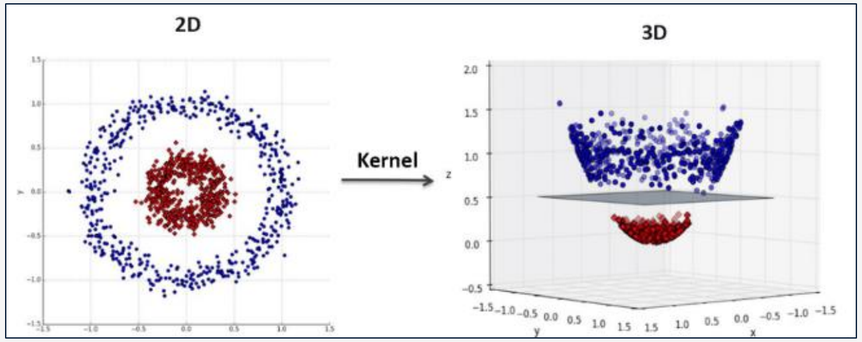
- Classi sbilanciate
- Alta confusione
- Accuracy discreta
- Scarse metriche per i QSO

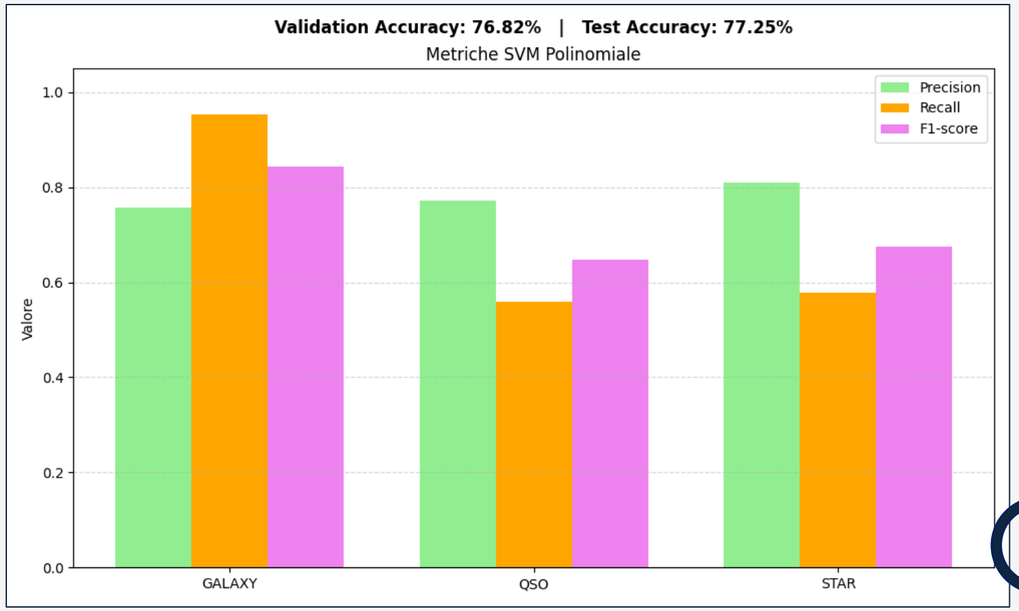
La classe predominate è favorita, la minoritaria penalizzata. Approccio lineare inadatto.

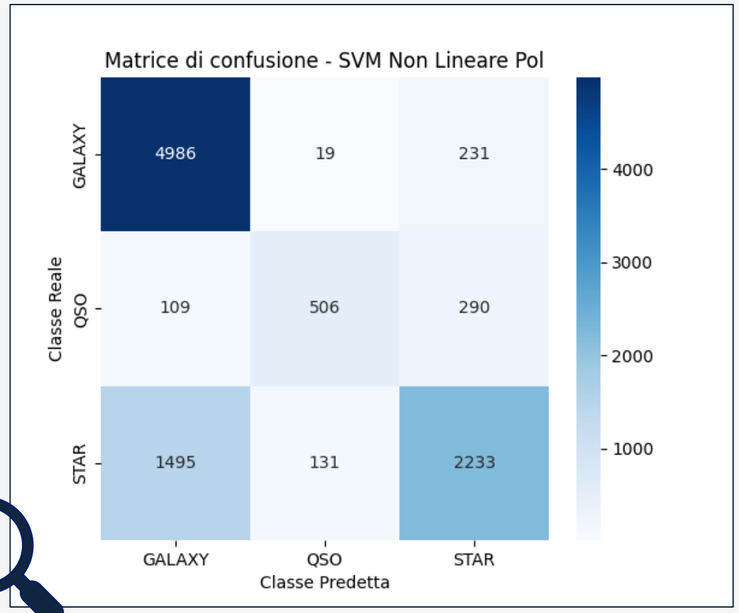
SVM polinomiale

Trasformazione con **kernel non lineare**

```
model = SVC(kernel='poly', degree=3, C=1.0, gamma='scale', coef0=0.0,
decision_function_shape='ovo', random_state=42)
```







Validation-accuracy: 76.8%
Test-accuracy: 77.3%

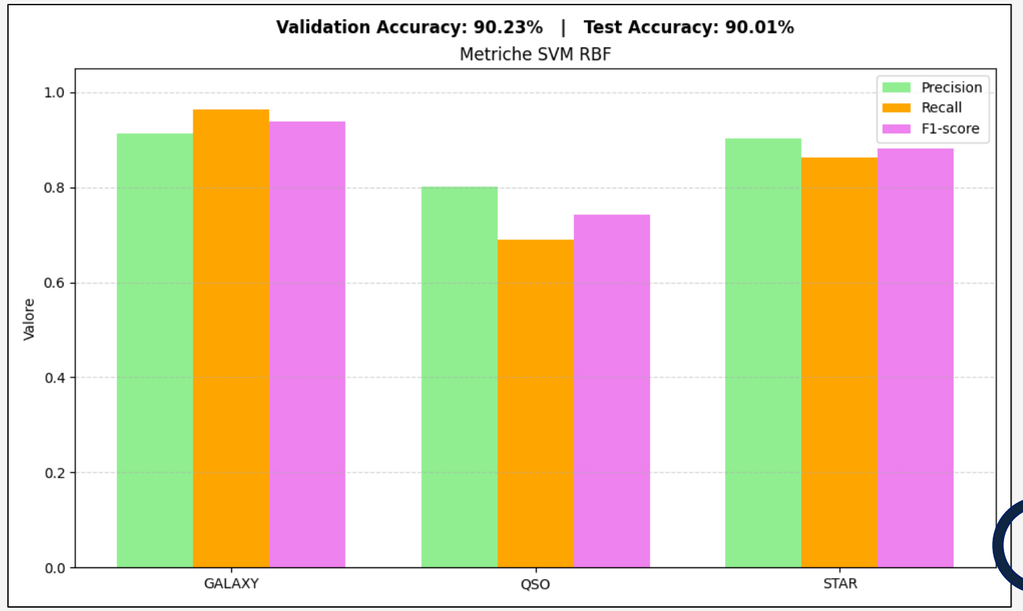
- Minor confusione per i QSO
- Confusione generale ancora alta
- Accuracy non migliorata
- Metriche ancora non sufficienti

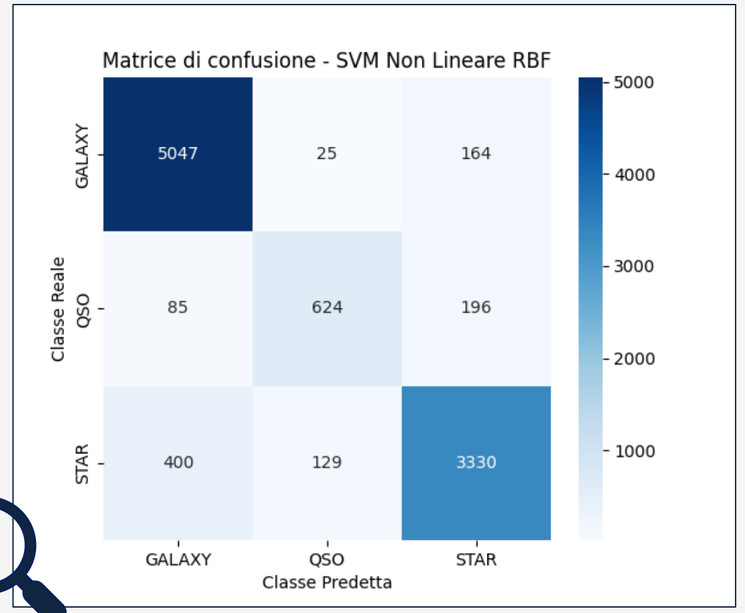
Ottimizza una specifica classe a discapito delle altre.
Non offre un vantaggio netto.

SVM radial basis function

Trasformazione con **kernel non lineare**

```
model = SVC(kernel='rbf', C=1.0, gamma='scale',  decision_function_shape='ovo',
random_state=42)
```





Validation-accuracy: 90.2%
Test-accuracy: 90%

- Buona accuracy
- Migliori risultati per tutte e tre le classi, leggermente inferiori per i QSO

Buon compromesso tra accuratezza complessiva e robustezza per classe.
Tuttavia, c'è ancora margine di miglioramento.

Ottimizzazione 1

Eseguita con **Grid Search**

```
param_grid = [
        {'kernel': ['linear'], 'C': [0.1, 1, 10, 100]},
        {'kernel': ['rbf'], 'C': [0.1, 1, 10, 100], 'gamma': [1e-3, 1e-2, 0.1, 1]},
        {'kernel': ['poly'], 'C': [0.1, 1, 10], 'degree': [2, 3, 4], 'gamma': ['scale', 'auto']},
    ]
model = SVC(decision_function_shape='ovo', random_state=42)
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy', verbose=2, n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)
best_model = grid_search.best_estimator_
best_params = grid_search.best_params_
best_kernel = best_params['kernel']
```

Set di valori da esplorare

- evita **underfitting** ed **overfitting**
- eseguita in modo **automatico**

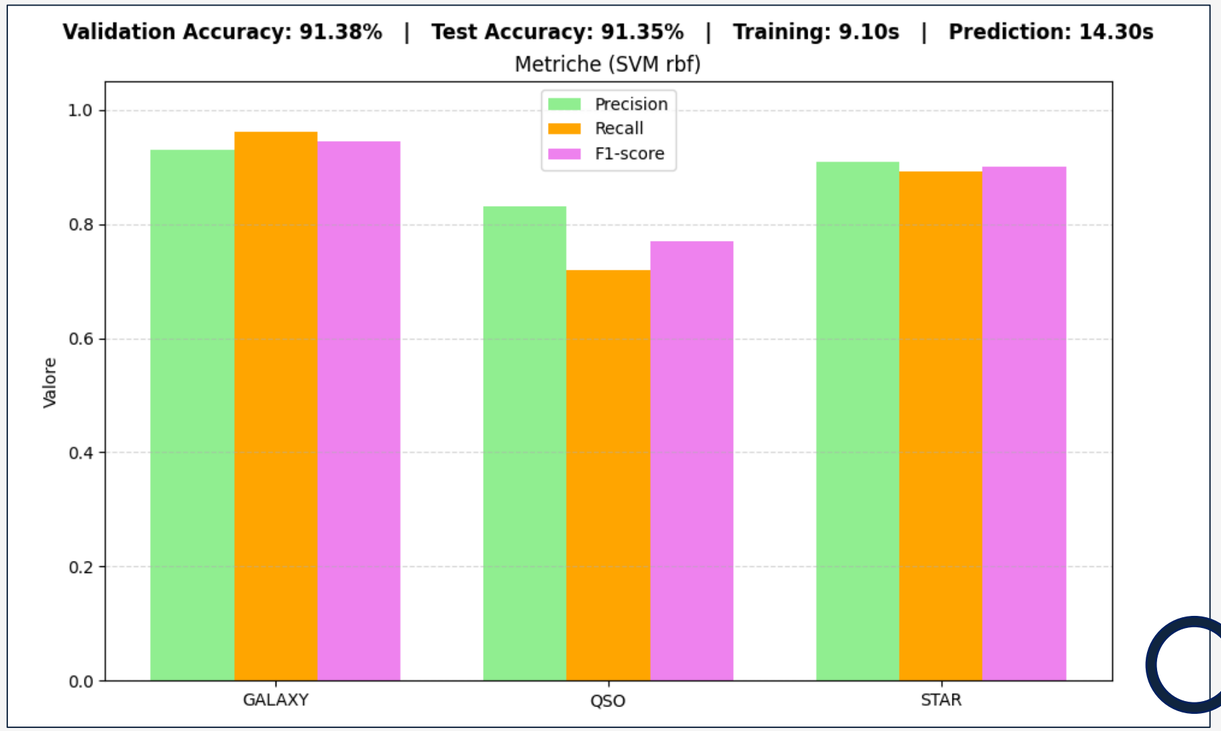
Otteniamo il **miglior modello** (kernel + iperparametri tarati)

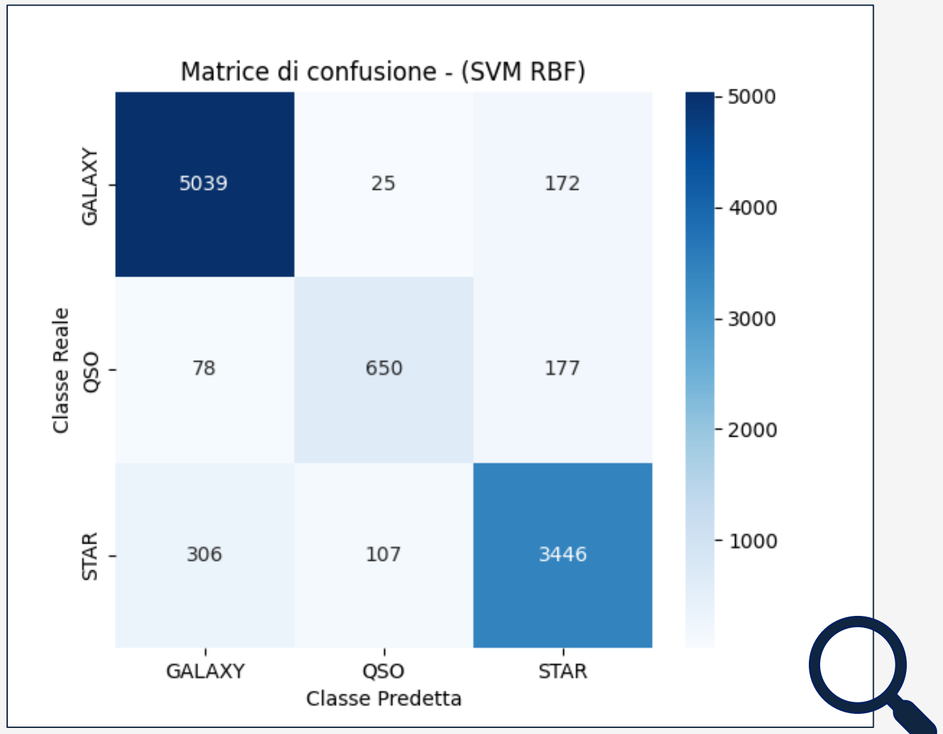
Miglior kernel	Parametri ottimali
rbf	C: 10, gamma: 1

Ottimizzazione 2

Miglior kernel	Parametri ottimali
rbf	C: 10, gamma: 1

Risultati ottenuti





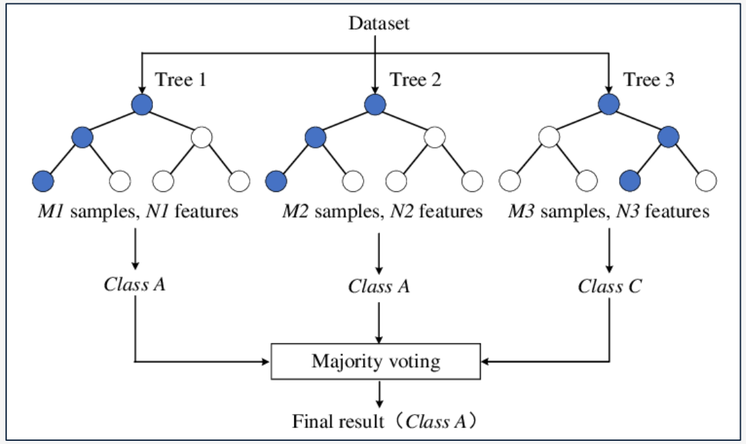
Validation-accuracy: 91.4%
Test-accuracy: 91.4%

Le metriche migliorano leggermente per i QSO, restando stabili per le altre classi. Impatto moderato, ma equilibrio e robustezza migliorati.

Random Forest 1

Modello basato su **alberi decisionali**

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
param_grid = {
        'n_estimators': [50, 100, 150, 200],
        'max_depth': [None, 50, 100, 150]
    }
model = RandomForestClassifier(random_state=42)
grid_search = GridSearchCV(model, param_grid, cv=4, scoring='accuracy', verbose=2, n_jobs=-1)
grid_search.fit(X_train_scaled, y_train)
best_model = grid_search.best_estimator_
best_params = grid_search.best_params_
```



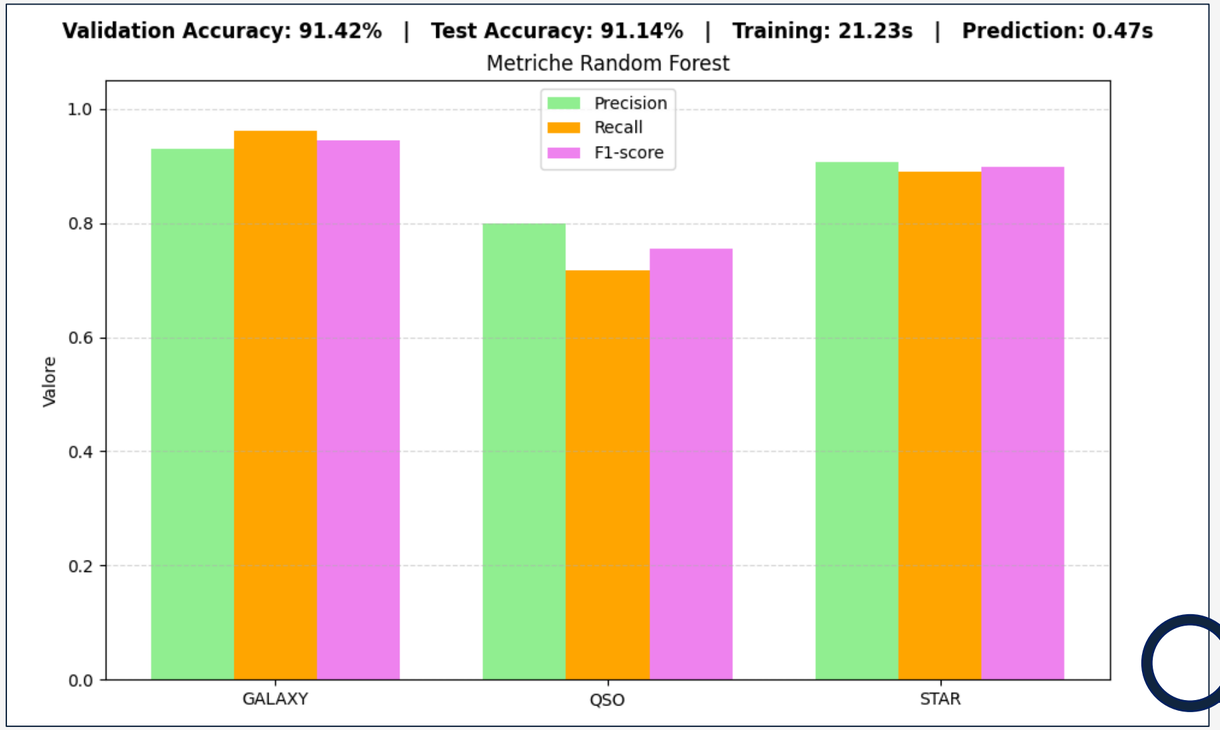
Parametri ottimali
n_estimators: 200 max_depth: None

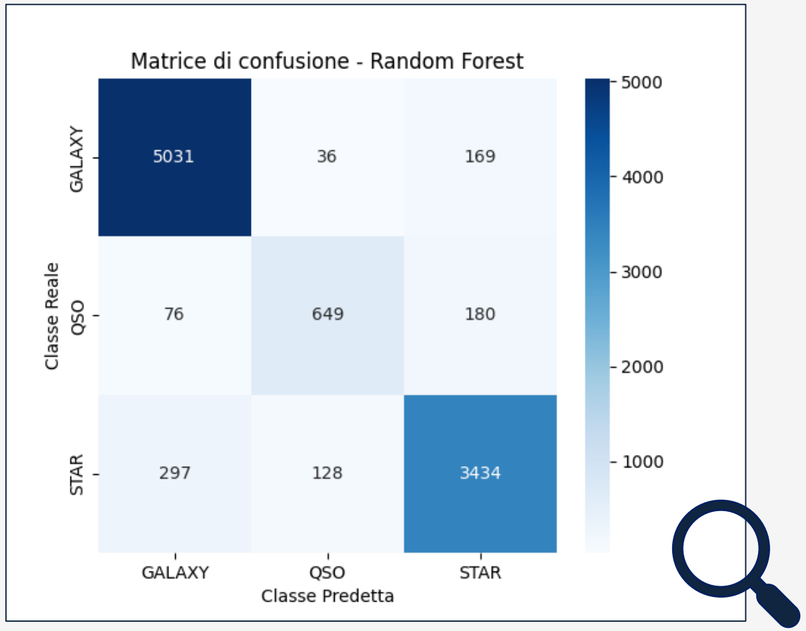
Altri parametri:

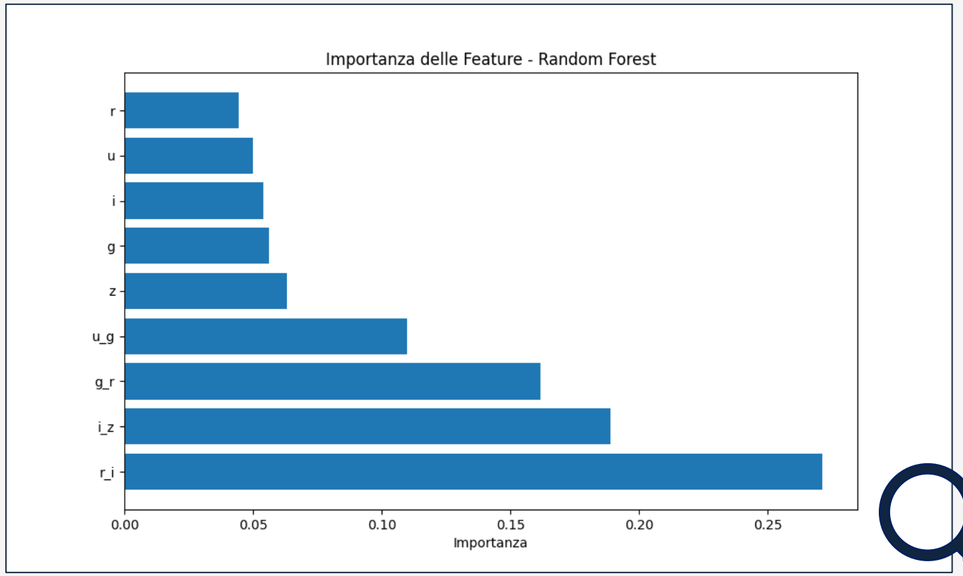
- **min_samples_split**
- **min_samples_leaf**
- **max_features**
- **bootstrap**

Random Forest 2

Risultati ottenuti





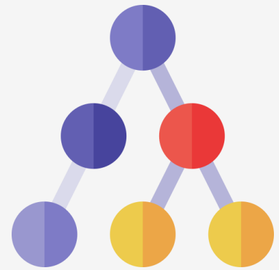


Performance metriche solide.
Riesce a gestire in modo efficace le classi sbilanciate.

Confronto

Performance simili in termini di accuratezza, ma...

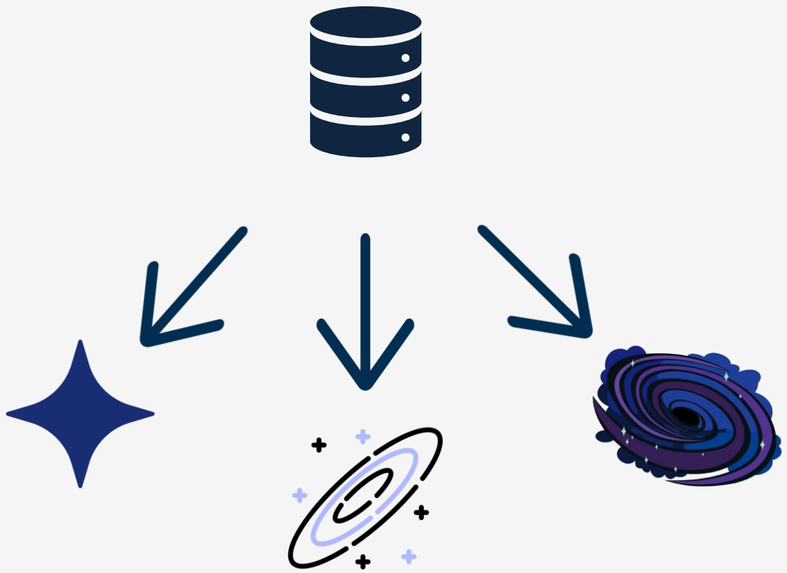
Modello	Validation accuracy	Test accuracy	Tempo di training	Tempo di predizione
SVM RBF	91.38%	91.35%	**9.10s**	**13.49s**
Random Forest	91.42%	91.14%	**21.23s**	**0.47s**



Tempi differenti!

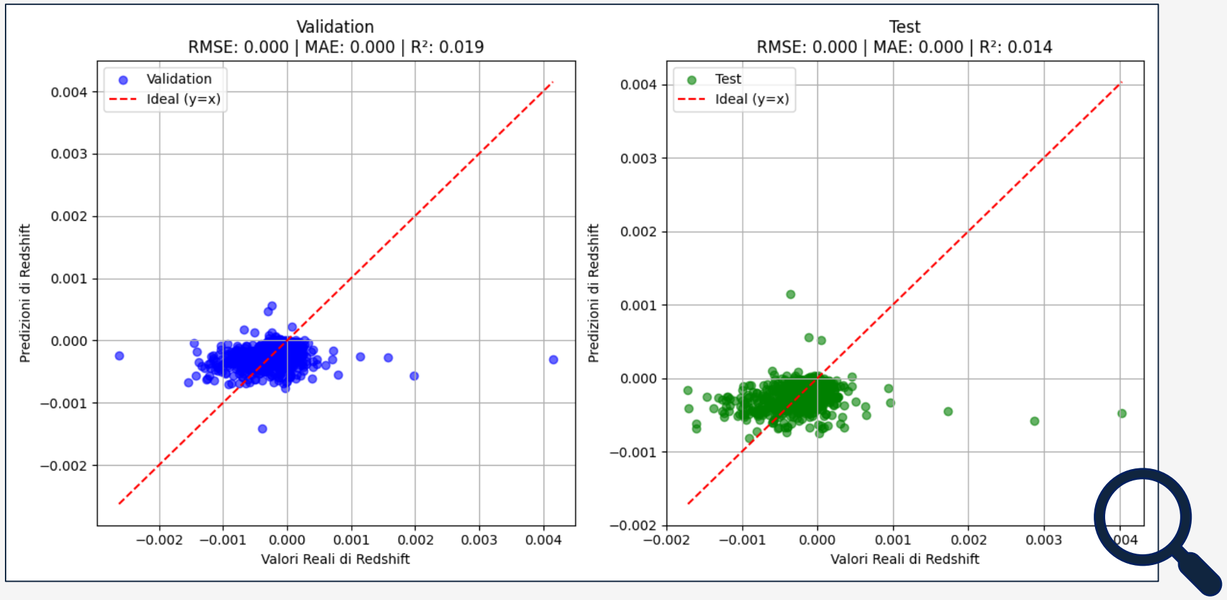
Redshift regression 1

Modus operandi



Applicazione modelli

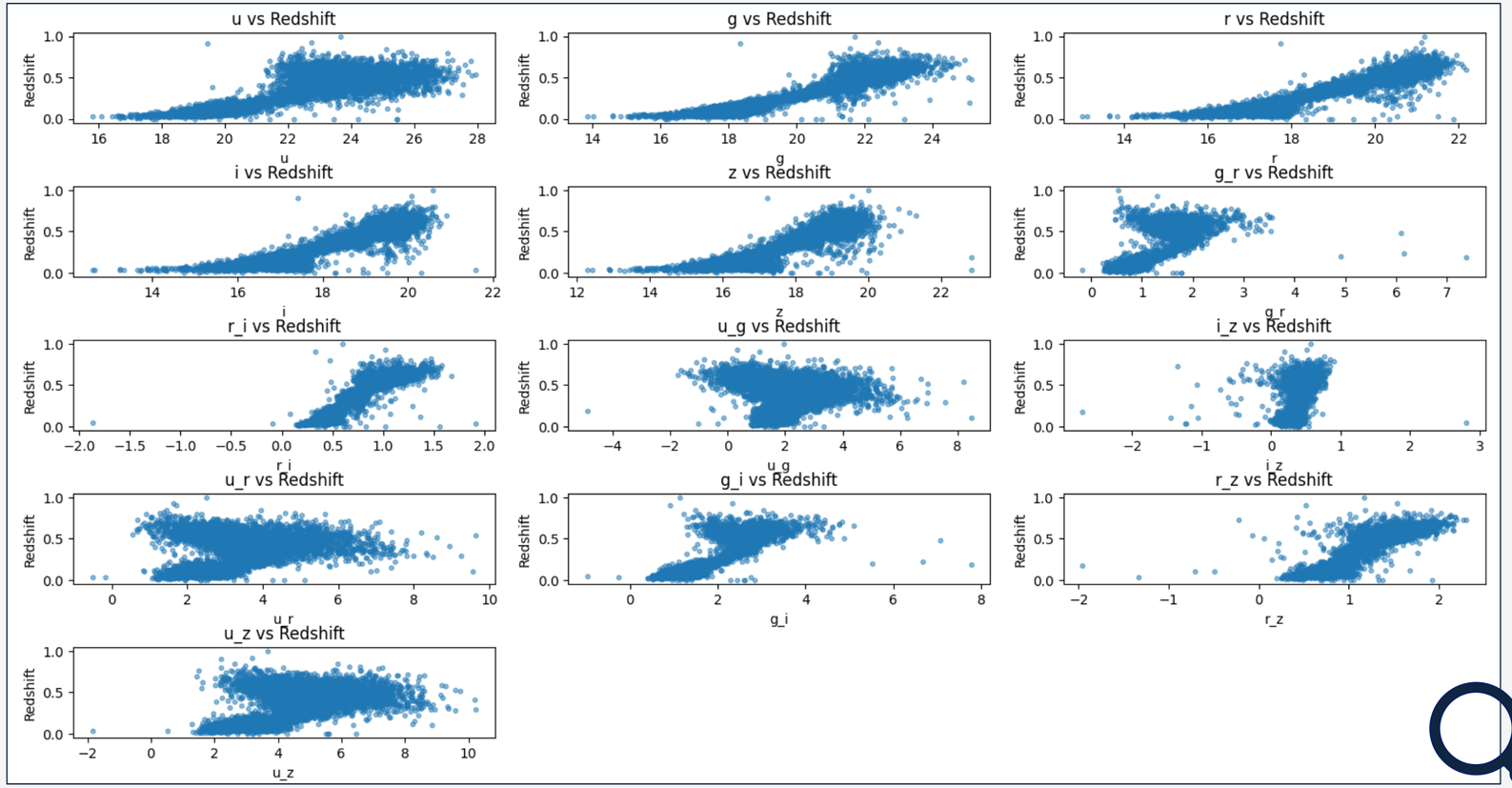
Per le **stelle**



- Redshift nullo
- **Non c’è necessità di applicare algoritmi di regressione.**

Redshift regression 2

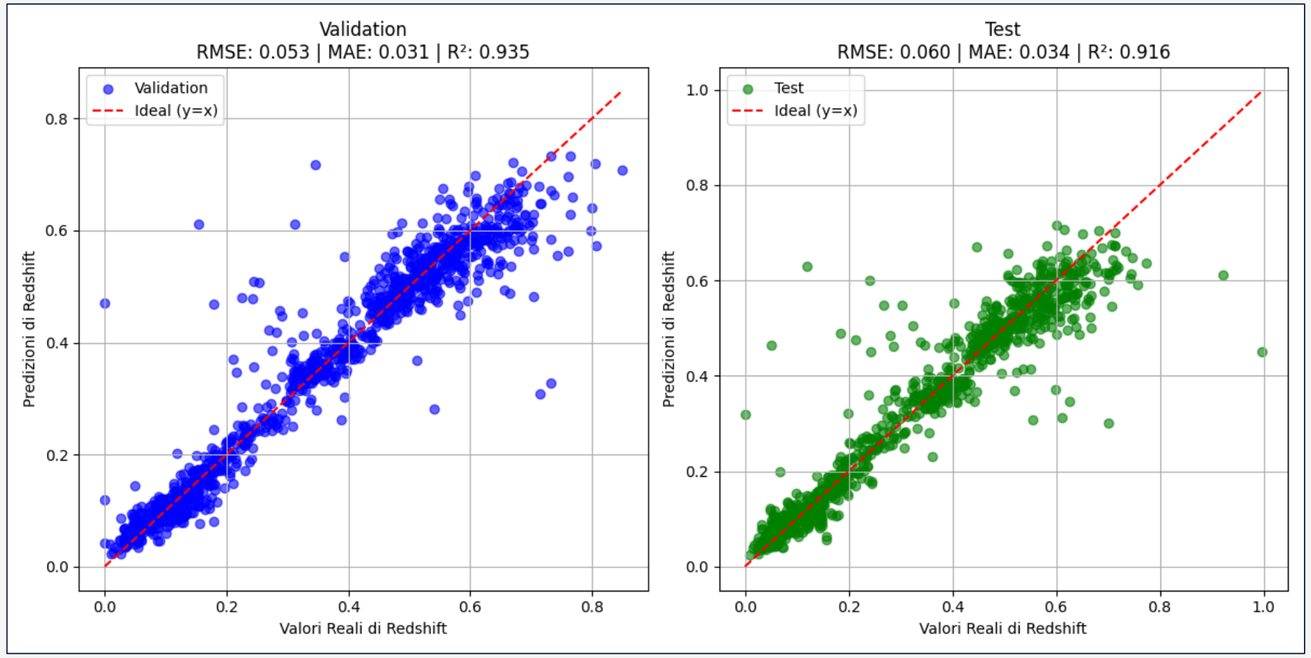
Per le **galassie**



Si nota una chiara relazione crescente tra le magnitudini e redshift

Redshift regression 2

Per le **galassie**

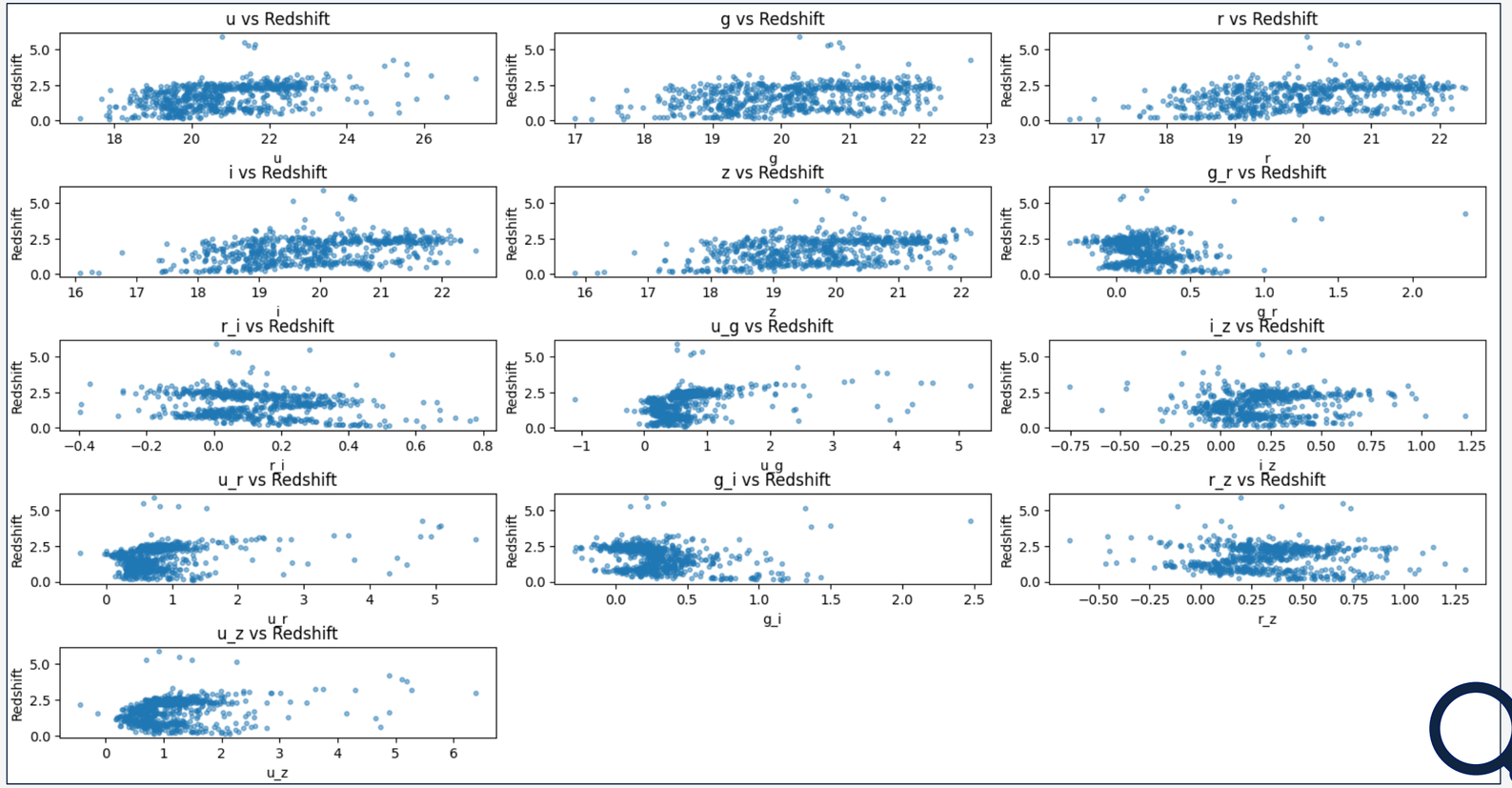


Modello	RMSE (val)	RMSE (test)	MAE (val)	MAE (test)	R^2 (val)	R^2 (test)
Lineare	0.062	0.065	0.043	0.046	0.913	0.904
Polinomiale	0.062	0.065	0.043	0.046	0.913	0.904
Lasso	0.062	0.065	0.043	0.046	0.913	0.904
SVR (RBF)	0.057	0.059	0.039	0.040	0.927	0.917
Random Forest	**0.053**	**0.056**	**0.031**	**0.033**	**0.935**	**0.926**

I modelli lineari sono una buona alternativa, ma la massima precisione si ha con Random Forest Regressor.

Redshift regression 3

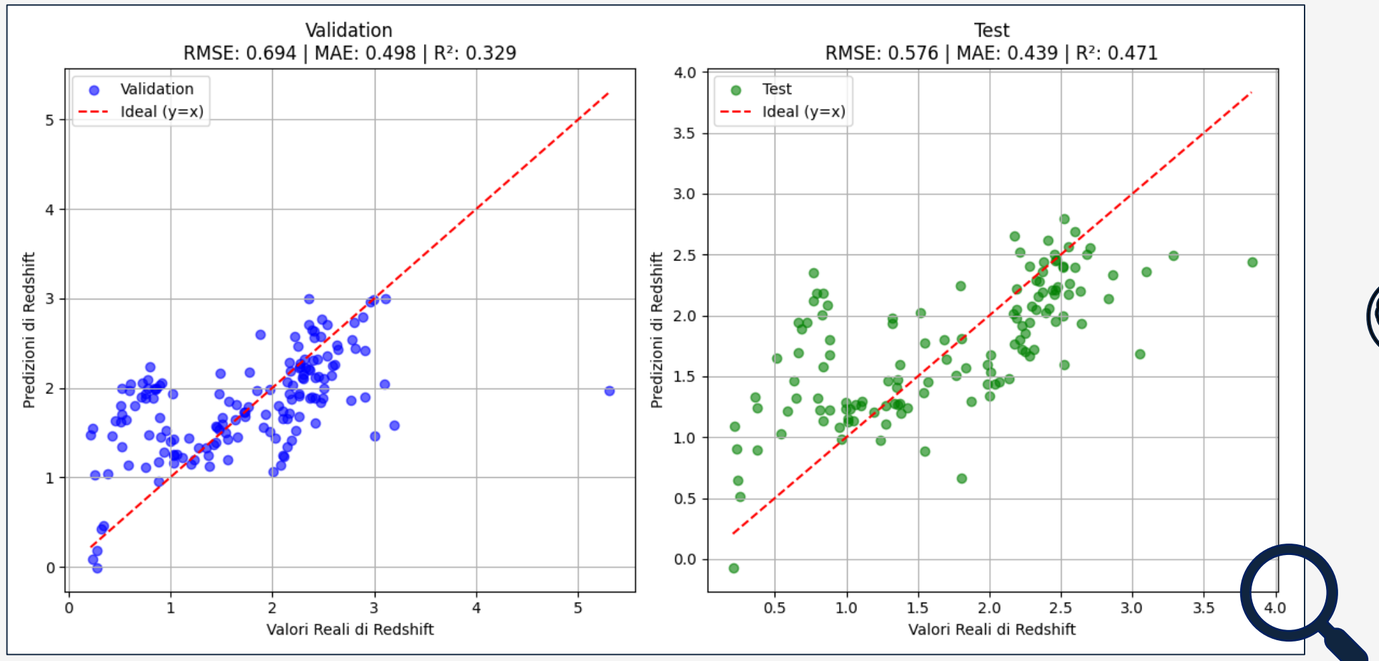
Per i **quasar**



Maggiore variabilità nei dati fotometrici, con relazioni più complesse

Redshift regression 4

Per i **quasar**

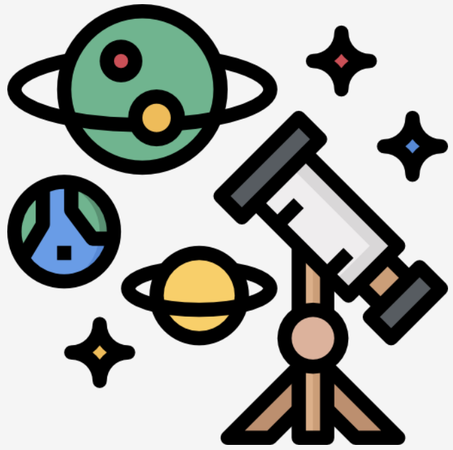


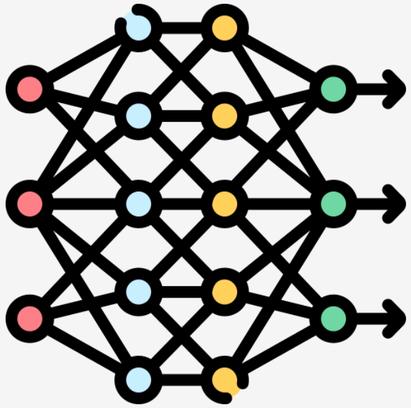
Altri modelli ottengono risultati peggiori dei Support Vector Regressor.
Difficile stimare il redshift fotometrico nei quasar.

Conclusioni

Importanza dell'**analisi**

Applicazioni per grandi **survey astronomiche**





Approcci di **deep learning**

Università
degli Studi
della Campania
Luigi Vanvitelli

15 Maggio 2025

Grazie per l'attenzione

gaetano.improta1@studenti.unicampania.it